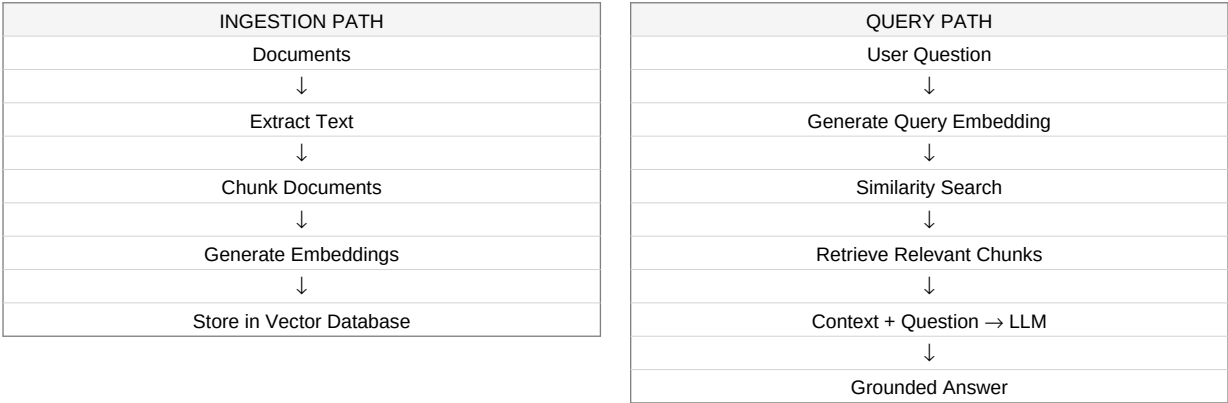


RAG Fundamentals — RAG Explained

Retrieval-Augmented Generation (RAG) grounds a large language model with relevant information retrieved from an external knowledge base. Instead of relying only on training knowledge, the model receives specific company information as context before generating an answer.



How RAG Works

During ingestion, documents such as FAQs, policies, manuals, product information, and internal documentation are converted into text. The text is divided into smaller chunks. Each chunk is converted into an embedding representing its semantic meaning, and the embeddings are stored in a vector database.

When a user asks a question, the question is converted into an embedding and compared with stored embeddings using similarity search. Relevant chunks are retrieved and supplied to the LLM as context. The model then generates an answer grounded in that retrieved information.

Chunking and Why Chunk Size Matters

Chunking means dividing a document into smaller pieces before generating embeddings. Chunk size affects retrieval accuracy and how much context is supplied to the model.

Chunk choice	Effect
Too small	Important context can be separated across different chunks.
Too large	Retrieved text may contain irrelevant information and consume more context.
Appropriate size	Keeps enough context to understand the information while remaining focused for accurate retrieval.

3 Real Problems RAG Could Solve in a Petalnex-Type Business

- 1. Internal Documentation Assistant** — Employees could ask questions about company policies, internship procedures, technical documentation, or internal processes and receive answers based on actual company documents.
- 2. Customer Support Knowledge Base** — Support staff could ask about products, services, troubleshooting procedures, FAQs, and company policies. RAG could retrieve relevant documentation before generating a response.
- 3. Employee and Intern Onboarding** — New employees or interns could query onboarding documents, project guidelines, tools, procedures, and frequently asked questions instead of repeatedly asking supervisors.

Key takeaway: RAG combines retrieval with generation so an LLM can answer using specific, relevant knowledge rather than relying only on pretrained knowledge.